Token Averaging for Compute-Efficient Language Model Pretraining

Working draft: results are being added as experiments complete

Vardh*

July 2026

Abstract

The dominant cost of language model pretraining is proportional to the number of transformer positions processed: every raw token occupies one position, so seeing more data costs proportionally more compute. We study *token averaging*, a simple modification in which every k consecutive token embeddings are mean-pooled into a single transformer position before the first attention layer, so that a model processes $k\times$ more raw text for approximately the same compute. A central obstacle is that a k-averaged model natively scores only one token per window, making its loss incomparable to a standard model's. We address this with an *offset-ensemble* evaluation that recovers a true full-sequence negative log-likelihood at approximately the cost of one baseline forward pass, and we verify at 125M parameters that the procedure is valid even for a checkpoint trained without offset randomization. On this strictly comparable metric, a 125M model with $2\times$ averaging improves perplexity by 3.4% over its baseline while spending $0.91\times$ the training FLOPs. On native training losses, the compute needed to match baseline loss shrinks by $1.15\times$ at 50M and $1.44\times$ at 125M; the apparent growth of this multiplier with scale is suggestive but so far confirmed on the comparable metric only at 125M, and 250M runs are in progress. A controlled decomposition indicates the gain comes from cheaper data throughput rather than from extended context: within each method, spending the same savings on longer raw context hurts on packed web text at small scale, and a preliminary comparison suggests that when raw context is the goal, averaging obtains it more cheaply than widening the attention window. Finally, a sweep over pooling functions under a matched protocol shows that, among the rules tested, the best within-window pooling depends on the compression ratio: content-dependent learned weights lead at k=2, while a fixed recency-weighted rule wins clearly at k=4; matched mean-pooling controls are still training. [TODO: finalize numbers at 250M; add matched-control ablation results; tighten to venue length]

Contents

*[TODO: author list, affiliations, contact]

1 Introduction

Large language models are trained by next-token prediction over enormous corpora, and the arithmetic of that training is unforgiving: with standard architectures, each raw token of training data occupies exactly one position in the transformer, so the compute bill scales linearly with the amount of data seen (and, through the attention term, superlinearly with context length). Compute-optimal scaling analyses [TODO: cite Hoffmann et al. 2022] imply that most of a training budget should be spent on data rather than parame which makes the *cost per token of data* the single most important constant in the training economy. Any mechanism that lowers it without proportionally degrading what the model learns per token converts directly into lower loss at fixed compute, or equal loss at lower compute.

This paper studies what is arguably the simplest such mechanism: *token averaging*. Before the first transformer layer, we partition the input sequence into non-overlapping windows of k consecutive tokens and replace each window's k embedding vectors by their mean. The transformer then runs on T/k positions instead of T, and is trained to predict the first token of the following window at each compressed position. Nothing else changes: no new parameters are introduced (the pooling is a fixed mean), and the architecture, tokenizer, optimizer, and data pipeline are identical to the baseline. The hypothesis is that transformers can model mildly compressed input representations nearly as well as raw ones, in which case a k-averaged model can afford to train on $\sim k\times$ more raw data at the same compute budget and end up strictly ahead.

The idea invites immediate skepticism on two grounds, and much of this paper is devoted to answering both carefully.

Is the comparison even well-posed? A k-averaged model, as trained, assigns probabilities to only one token per window: with k=2 it predicts the tokens $t_3, t_5, t_7, \ldots$ (the first token of each window, given all preceding windows) and simply has no conditional distribution for the remaining positions. Its training loss is therefore an average over roughly *half* the terms that make up the baseline's loss, and the two numbers are not measurements of the same quantity. A reviewer's summary would be blunt: "your model matches the baseline on the half of the task it attempts." We resolve this with an *offset-ensemble evaluation* (Section 3.6): the model is run k times per evaluation sequence with the window grid shifted by $o = 0, \ldots, k-1$ tokens, so that every position beyond the first window is predicted exactly once from its full compressed prefix. This recovers a genuine full-sequence negative log-likelihood at approximately the cost of a single baseline forward pass, and it suggests a rotating-offset decoding scheme for token-by-token generation (a design we describe but have not yet implemented or benchmarked). We also verify empirically, at 125M, that the evaluation is valid for a checkpoint trained without any offset randomization, and we explain why sequence packing makes that expected.

Where would a gain even come from? The FLOPs saving from halving the number of positions is smaller than it first appears, and it pays to state the accounting carefully. At a fixed number of raw tokens, averaging with factor k cuts the cost of the linear terms (attention projections and the feed-forward network) per raw token by k, because those terms are proportional to the number of positions processed. But the deployment we study trains on $k\times$ more raw tokens at the same budget, which restores the linear-term total exactly; only the attention term, whose per-position cost also depends on sequence length, ends up genuinely lower. That residual saving is the attention share of total compute, which falls with model width from roughly 13% at our 50M configuration to a few percent at billion-parameter widths (Section 3.5). The interesting question is therefore not whether the matmuls get cheaper (they barely do) but whether *embedding twice the data into the*

same transformer budget teaches the model more than processing half the data at full resolution. Equivalently: the method purchases data throughput per FLOP, and possibly extended raw context, and we want to know what each of those is worth. We design two deployment configurations that decompose exactly this (Section 3.4): Config A holds the raw sequence length fixed and pockets the compute saving, while Config B holds the transformer length fixed and spends the entire saving on $k\times$ longer raw context. A fourth a standard model with a genuinely widened attention window, controls for whether any cont effect is specific to averaging.

Findings. Our main experimental findings, on FineWeb with GPT-NeoX-style models trained from scratch, are as follows. We state for each finding which metric supports it, because the two metrics in play (native eval loss and the strictly comparable full-position NLL) have different evidential standing.

1. **At 125M, the gain holds on a strictly comparable metric.** Under offset-ensemble evaluation, with identical evaluation sequences and identical target positions (3,409,119 predictions each), the k=2 averaged model scores 4.141 nats/token versus 4.177 for the baseline: 3.4% lower perplexity at 0.91× the training FLOPs. Strikingly, the model's per-offset losses are essentially equal (4.145 at offset 0 vs. 4.138 at offset 1) even though it was nominally trained only at offset 0; packed training data effectively randomizes the window grid relative to the text, which explains the robustness (Section 5.2).

2. **On native losses, iso-FLOP endpoints favor averaging at every scale measured, with a suggestive scale trend.** Reading the loss-versus-FLOPs curves horizontally, the baseline needs 1.44× more compute at 125M, and 1.15× at 50M, to reach the loss the averaged model reaches. These multipliers are computed from native losses, which compare different position sets across k; the 125M comparison is the one validated by finding 1, and full-position rescoring of the 50M and 250M checkpoints is queued. Until then we treat the growth of the multiplier with scale as suggestive rather than established (Section 5.1).

3. **The mechanism appears to be data-per-FLOP, not context.** In the decomposition experiment at 50M, every *within-method* comparison says extended raw context hurts on packed FineWeb: widening a standard model's attention window from 1024 to 2048 costs +0.214 nats at 1.26× the compute; within k=2, spending the averaging saving on doubled raw context (Config B) rather than pocketing it (Config A) costs +0.217 nats; within k=4 the corresponding penalty is +0.440 nats. Since added context is worthless to harmful here, Config A's gains cannot be a context effect. A further observation, currently suggestive only: the averaged and full-attention 2048-context arms end at nearly identical native losses despite a 1.26× compute difference, hinting that averaging is the cheaper route to raw context; because those two losses are measured on different position sets, this claim awaits offset-ensemble rescoring (Section 5.3).

4. **Among the pooling rules tested, the best one depends on the compression ratio.** Under a matched training protocol at 50M, a content-dependent learned weighting (513 parameters) leads at k=2, while at k=4 a *fixed* exponential recency weighting beats the content-only learned scorer by a persistent 0.18 nats. Matched mean-pooling controls are still training, so no claim is made yet against plain mean pooling. We attribute the k=4 result to the learned scorer being content-aware but position-blind, and introduce a position-aware learned pooler intended to subsume both regimes (results pending; Section 5.4).

Contributions. (1) A systematic, FLOP-controlled study of token averaging as a *permanent* operating point, not a warm-up phase, across three model scales, with training budgets set for iso-FLOP endpoint comparisons. (2) The offset-ensemble evaluation protocol, which makes patch-style models comparable to token-level baselines on full-sequence likelihood and applies retroactively to checkpoints trained at a fixed offset (verified at 125M); together with a matching random-offset training scheme that removes the position-coverage asymmetry at the source, and a rotating-offset decoding design for generation (not yet benchmarked). (3) A decomposition showing that within each method, extended raw context hurts on packed web text at small scale, so the benefit of averaging there is cheaper data throughput; plus preliminary evidence that averaging is the cheaper route to raw context when context is wanted. (4) A pooling-function ablation under a matched protocol revealing a compression-dependent crossover between learned content-based weighting and fixed recency weighting, and a position-aware learned pooler designed to subsume both regimes.

2 Related Work

[TODO: This section is deliberately deferred. Anchor points to cover when we write it:]

- **Patch-level training.** Shao et al. (2024; ICLR'25), "Patch-Level Training for Large Language Models": the closest prior work. It averages K token embeddings into patches and predicts the next patch, but uses patch-level training only as a *warm-up* followed by token-level fine-tuning, at 370M–2.7B scale. Our deltas: patch-level as the final operating point (made comparable by the offset-ensemble evaluation), FLOP-controlled scaling-law treatment rather than single budget points, and the context-extension decomposition.
- **Hierarchical / byte-level patching.** MegaByte, Byte Latent Transformer (dynamic patch boundaries), Block Transformer, Funnel-Transformer, Hourglass. These change the architecture to compress and re-expand; we deliberately keep the architecture untouched.
- **Multi-token prediction.** Gloeckle et al. 2024; DeepSeek-V3's MTP objective. Related through the phased multi-token variant we briefly explore, but MTP predicts multiple targets from uncompressed inputs; we compress inputs.
- **Context compression.** Gist tokens, AutoCompressor, ICAE: these compress *prompts* at inference; we compress *all* input during pretraining.
- **Scaling laws.** Hoffmann et al. 2022 (Chinchilla) for the compute-optimal framing and the $D^* \approx 20N$ heuristic we use to set budgets; Kaplan et al. 2020.
- **Tokenizer granularity.** Work on tokenizer vocabulary size and bits-per-byte comparisons, relevant to the coarser-tokenizer control we plan (Section 6).

3 Method

3.1 Token averaging

Let $t_1, \ldots, t_T$ be a sequence of raw tokens and $e_i \in \mathbb{R}^{d_{\text{model}}}$ their (learned) input embeddings. For a fixed averaging factor k, we partition the sequence into $n = \lfloor T/k \rfloor$ non-overlapping windows of k

consecutive tokens and compress each window to a single vector by mean pooling:

$$\bar{e}_j = \frac{1}{k} \sum_{i=(j-1)k+1}^{jk} e_i, \qquad j = 1, \ldots, n. \tag{1}$$

The transformer body (attention layers, feed-forward layers, RoPE, final norm) runs unchanged on the compressed sequence $\bar{e}_1, \ldots, \bar{e}_n$, producing n output states, and the output head produces logits over the vocabulary at each compressed position. With k=1 the method reduces exactly to the standard language model. Mean pooling introduces *no new parameters*; Section 3.7 describes the alternative pooling functions we ablate, the largest of which adds 517 parameters.

Positional information deserves a comment. Rotary position embeddings are applied inside the transformer body to the *compressed* positions $1, \ldots, n$, so adjacent transformer positions are always k raw tokens apart. The geometry of the compressed sequence is therefore identical regardless of where the window grid is anchored in the raw text, a fact that becomes important for the offset analysis in Section 3.6.

3.2 Training objective

At compressed position j (windows and positions 1-indexed throughout) the model is trained to predict the *first token of the next window*: the target at position j is t_{jk+1}. Concretely, for k=2 on eight raw tokens:

raw tokens	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
averaged input	$\bar{e}_1 = \frac{e_1+e_2}{2}$		$\bar{e}_2 = \frac{e_3+e_4}{2}$		$\bar{e}_3 = \frac{e_5+e_6}{2}$		$\bar{e}_4 = \frac{e_7+e_8}{2}$	
target	$\to t_3$		$\to t_5$		$\to t_7$		(none)	

The loss is the standard cross-entropy averaged over the $n-1$ predicted positions. Note two properties that drive much of the paper. First, the model predicts only the tokens $t_{k+1}, t_{2k+1}, t_{3k+1}, \ldots$, one per window; the conditionals for all other positions are simply absent from the model as trained, which is the root of the evaluation comparability problem (Section 3.6). Second, each prediction conditions on a *lossy* summary of the past: the most recent window contributes only through its mean, so fine within-window order information is discarded. How much this costs, and whether smarter pooling recovers it, is the subject of Section 5.4.

We also experimented with a phased "token superposition" variant in which, for the first fraction of training, each compressed position predicts *all* k tokens of the next window as a bag (multi-hot cross-entropy through the single LM head), before reverting to standard single-token prediction. We treat it as an auxiliary experiment rather than part of the main method and report it only in the appendix. [TODO: decide whether phased results are in or out of the paper]

3.3 Random-offset training

As trained above, the window grid is always anchored at the start of the sequence, so the model is nominally only ever asked to predict the raw positions $jk+1$, i.e. $t_{k+1}, t_{2k+1}, \ldots$ of each training sequence. Although sequence packing already randomizes the grid *relative to the underlying text* (Section 3.6), we additionally remove the asymmetry at the source in our current training code: at each training step we sample an offset $o \sim \text{Uniform}\{0, \ldots, k-1\}$ per batch, drop the first o tokens of the batch, and proceed as usual. Over many steps every residue class of positions receives equal training signal. At evaluation time the offset is fixed to 0 (or swept, for the offset-ensemble metric).

The change costs at most one window per sequence and touches nothing else. Checkpoints reported in this draft predate this change unless noted; the one checkpoint we have tested (125M, k=2) is empirically offset-robust despite that (Section 5.2).

3.4 Two deployment configurations: cheaper tokens vs. free context

At a fixed FLOP budget, the compute freed by averaging can be spent in two distinct ways, and distinguishing them turns out to be scientifically important. Throughout, `seq_len` denotes the number of *raw* tokens per training sequence and $L = \texttt{seq\_len}/k$ the number of positions the transformer actually processes.

Config A (cheaper tokens; the paper's main configuration). The raw `seq_len` stays at the baseline's value (1024). The transformer length shrinks to $L = 1024/k$, each raw token costs roughly $1/k$ of baseline, and the model trains on $k\times$ the raw tokens for the same endpoint FLOPs. The raw context available to any prediction is unchanged (1024 tokens); only its representation is compressed.

Config B (free context). Scale `seq_len` up to $k\times 1024$ so that $L = 1024$ is *identical* to the baseline. Now each sequence embeds $k\times$ the raw context into the same transformer computation ("double context for free"), and per raw token the cost is again $1/k$ of baseline, giving iso-FLOPs at $k\times$ tokens by construction.

Both configurations use identical models, data, and optimization; they differ only in where the savings go. To determine whether any Config B effect is specific to averaging, we add a control arm: a standard k=1 model with a genuinely widened attention window ($\texttt{seq\_len} = L = 2048$), which obtains the same raw context the traditional way at 1.26× the baseline's per-token cost.

An honest historical note: Config A originated as a bug. The first k=2 runs were accidentally launched with the same raw `seq_len` as the baseline rather than the doubled value we had intended. The accident turned out to be the cleaner experiment, since it holds raw context fixed and isolates the data-per-FLOP effect, and it became the paper's primary configuration, with the originally intended setting preserved as Config B.

3.5 FLOPs accounting

We recompute all FLOPs from first principles rather than trusting logged values. For a transformer with hidden size d, n_{layers} layers, and transformer sequence length L, our architecture (attention with Q, K, V, output projections; SwiGLU FFN with expansion factor 2.5) costs, per layer and per position, $23d^2 + 4Ld$ multiply–accumulate FLOPs forward: $8d^2$ for the four attention projections, $15d^2$ for the SwiGLU FFN (see Appendix A for the exact derivation), and $4Ld$ for the attention score and value aggregation terms. With the usual 3× multiplier for forward-plus-backward, a training run that consumes D raw tokens at raw sequence length `seq_len` costs

$$\text{FLOPs}(D) = \frac{D}{\texttt{seq\_len}} \cdot n_{\text{layers}} \cdot L \cdot 3\left(23d^2 + 4Ld\right), \qquad L = \texttt{seq\_len}/k. \tag{2}$$

Equation (2) makes the economics of Config A transparent, and it is worth separating two comparisons that are easy to blur. *At a fixed raw-token count* D, averaging processes D/k positions instead of D, so the linear-term cost ($23d^2$ per position) per raw token falls by a factor of k, and the attention term falls by k^2 (a factor k from fewer positions and another from $L \to L/k$). *At the iso-FLOP deployment*, however, the averaged run consumes kD raw tokens, which restores

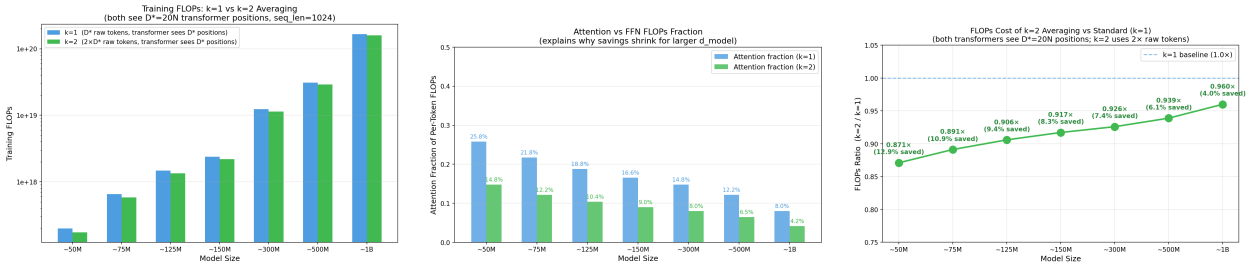


Figure 1: FLOPs structure across model widths. **Left:** absolute training FLOPs per raw token. **Middle:** the attention share of per-position compute shrinks with d_{model}. **Right:** consequently, the raw per-token FLOPs ratio of k=2 averaging to baseline at fixed raw `seq_len` approaches 1 as models widen: the direct arithmetic saving is small and shrinking, which is why we frame the method as buying data throughput, not cheaper matmuls.

the linear-term total to exactly the baseline's; only the attention term remains lower, because its per-position cost depends on the shorter L. The net arithmetic saving at the iso-data multiple is therefore just the attention share of total FLOPs, which at L=1024 falls from ∼13% at d=512 to ∼4% at d=2048 (Figure 1). We emphasize this deliberately: the value proposition of token averaging is not cheaper matrix multiplications. It is that $k\times$ *more raw data flows through the same transformer budget*, and the empirical question is what that data throughput is worth in loss.

3.6 The evaluation problem and the offset-ensemble fix

The problem. A language model's defining scalar is the full-sequence log-likelihood $\sum_i \log p(t_i \mid t_{<i})$. The k=1 baseline's eval loss is the mean of all $T-1$ such terms per sequence. The k-averaged model's eval loss, as trained, is the mean over only the $\sim T/k$ positions it predicts; the remaining conditionals do not exist in the model. The two losses are therefore averages over *different sets of terms*, and no amount of extra evaluation data fixes this: evaluating the averaged model on more batches shrinks the error bar of the wrong quantity. (We record this dead end explicitly because it was our own first fix attempt: matching the *count* of predicted tokens by evaluating on $k\times$ the batches matches sample sizes, not tasks.) An additional casualty is generation: a model that predicts one token per window cannot decode token-by-token as trained.

The fix. For evaluation, run the model k times over each sequence, shifting the window-grid anchor by $o = 0, 1, \ldots, k-1$ raw tokens. At offset o, windows cover tokens $(o{+}1..o{+}k), (o{+}k{+}1..o{+}2k), \ldots$ and the model's targets are the raw positions $o + jk + 1$ for $j = 1, 2, \ldots$, i.e. $t_{o+k+1}, t_{o+2k+1}, \ldots$ Across the k offsets these target sets are disjoint, and their union is every position $> k$ (all residue classes are covered), each predicted exactly once conditioned on its full compressed prefix. This yields a genuine full-sequence NLL over all scored positions, directly comparable to the baseline's on the same positions. The cost is k forward passes over sequences of length $L = T/k$, approximately *one* baseline forward pass in total, since per-pass cost scales with L. For an airtight comparison we additionally score the baseline restricted to the identical set of target positions, eliminating even the residual composition difference from the first k tokens. The same construction suggests a decoding scheme for generation: rotate the offset during decoding so that a window boundary always ends at the current position, maintaining k rotating KV caches whose amortized cost matches one baseline decoder. We describe the scheme in Appendix B; implementing and benchmarking it is future work, so all generation-related statements in this paper are architectural arguments, not measurements.

Why it can work on checkpoints trained at a fixed offset. One would expect a model trained only at offset 0 to be miscalibrated at other offsets. For the one checkpoint we have tested (125M, k=2) it is not (Section 5.2), and in hindsight the reason is structural. Training sequences are produced by packing a continuous token stream into fixed-length chunks whose boundaries fall at arbitrary points of the underlying documents; consequently the window grid's alignment *relative to the text* was effectively uniformly random throughout training, even though it was always "offset 0" relative to the chunk. Moreover, RoPE operates on compressed positions (Section 3.1), so all offsets induce identical positional geometry. This argument applies equally to our other checkpoints, but we treat offset robustness as verified only where measured, and we will check it as each checkpoint is rescored. The same reasoning motivated the explicit random-offset training of Section 3.3 as a formalization of what packing already does, rather than as a behavioural change.

3.7 Pooling functions

Mean pooling (Eq. 1) is the simplest possible aggregation, and everything upstream of the transformer flows through it. We therefore ablate the pooling function while holding all else fixed. All variants below compress k (or a window of) embeddings to one vector and leave the rest of the pipeline untouched.

Mean (default). Equal weights $1/k$. No parameters.

Fixed weighted (exponential). $\bar{e} = \sum_i w_i e_i$ with $w_i \propto \exp(\alpha i)$, α set so the last-to-first weight ratio is 20, then L_1-normalized. A pure recency prior: the most recent token in a window dominates. No learned parameters. (We also implemented linear, Gaussian, and triangular schemes; the exponential scheme is the representative we trained.)

Learnable (content-based). A single shared linear scorer $s_i = w^\top e_i + b$ produces one scalar per token; weights are $\alpha = \mathrm{softmax}(s_{1..k})$ within each window and $\bar{e} = \sum_i \alpha_i e_i$. This adds $d_{\text{model}} + 1 = 513$ parameters, zero-initialized so training starts exactly at mean pooling. The scorer sees each embedding independently, *before* any positional information exists in the network: it is content-aware but position-blind, a property that turns out to matter (Section 5.4).

Learnable (content + position). The same scorer plus a learned per-slot bias: $s_i = w^\top e_i + b + p_i$ with $p \in \mathbb{R}^k$ trained jointly ($d_{\text{model}} + 1 + k$ parameters). We initialize p at uniform for k=2 and at the exponential recency profile for k=4, i.e. at the best fixed rule found for each compression ratio, so the model starts from the strongest known prior and can adapt both terms end-to-end. (Runs pending.)

Overlapping windows. Average windows of w=4 tokens with stride s=2: the same 2× compression as k=2, but adjacent windows share tokens, so no token sits on a hard boundary. Targets are the first token beyond each window.

Word-aligned windows. Windows of variable size ($\geq k$, force- closed at $2k$) that end only at word boundaries, detected via the tokenizer's whitespace-prefix convention, so no window blends two words. Realized compression on FineWeb is ∼2.1× at k=2 (slightly above nominal), and the predicted positions are word starts; both caveats affect comparability and are handled explicitly (Appendix C).

4 Experimental Setup

Architecture. All models are GPT-NeoX-style decoder-only transformers built from the OLM implementation: pre-norm blocks, rotary position embeddings, SwiGLU FFN with expansion factor 2.5, and a padded vocabulary of 50,304 (tokenizer vocabulary 50,257). Table 1 lists the three scales. The 50M pair reported in the main comparison was trained with untied input/output embeddings; the 125M and 250M pairs tie them. Within every comparison the setting is identical across arms; across scales it is one of the residual inconsistencies we flag honestly in Section 6. [TODO: unify tying across scales in the final rerun matrix]

Table 1: Model configurations. "Raw tokens" are the training budgets for the k=1 / k=2 arms, set so both arms end at approximately equal total training FLOPs (Eq. 2); the k=1 budgets follow the Chinchilla $D^* \approx 20N$ heuristic. All main runs use raw `seq_len` = 1024 (Config A).

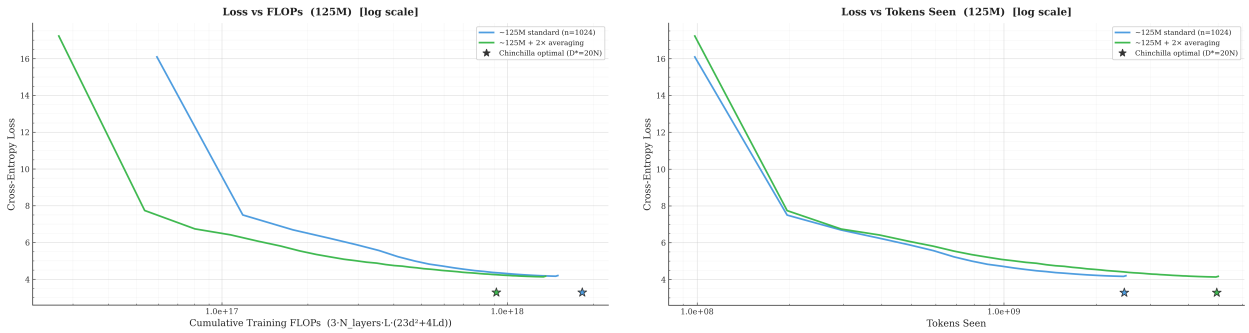
Scale	Params	d_{model}	Heads	Layers	LR	Raw tokens (k=1 / k=2)
50M	50.9M	512	8	8	2.0×10^{-4}	1B / 2B
125M	123.5M	768	12	12	1.6×10^{-4}	2.5B / 5B
250M	252.8M	1024	16	16	1.4×10^{-4}	5B / 10B

Data and tokenization. All models train from scratch on FineWeb (`sample-10BT`), tokenized with the EleutherAI Pythia-70m (GPT-NeoX) BPE tokenizer. The token stream is packed into fixed-length sequences without document-boundary masking; this is standard practice, but consequential for the context experiments (Section 5.3). Evaluation uses a held-out split of the same distribution, identical across all arms of a comparison.

Optimization. AdamW with a cosine learning-rate schedule: 2,000 warmup steps, decay to 10% of peak over exactly the run's total step count. Peak learning rates follow Table 1. Mixed-precision (bfloat16) training on NVIDIA GPUs via PyTorch DDP; gradient checkpointing at 250M. Batch sizes and step counts per run are tabulated in Appendix E, and we return to the optimizer-step confound they induce in Section 5.4.

Budgets and iso-FLOP design. For each scale the k=1 arm trains for approximately $20N$ raw tokens and the k=2 arm for twice that, which by Eq. (2) places the two endpoints at approximately equal training FLOPs (exactly equal in the linear terms; the k=2 arm actually ends *below* the baseline's FLOPs because its attention term is smaller: 0.87× at 50M, 0.91× at 125M). We report both the vertical reading (equal compute, compare loss) and the horizontal reading (equal loss, compare compute) of every curve, because late-training loss curves are so flat that small vertical gaps correspond to large horizontal ones.

Metrics. Two metrics appear in this paper and we are explicit about which supports which claim. "Native eval loss" is cross-entropy in nats per predicted token on held-out data at the model's native prediction positions (offset 0); it is the quantity logged during training, it is valid for comparing runs with the same k and window scheme, and it is *not* a comparable quantity across different k. The "full-position NLL" is the offset-ensemble metric of Section 3.6; it is comparable across k and is the paper's authoritative metric. At the time of writing the full-position metric has been computed for the 125M pair; rescoring the 50M and 250M checkpoints is queued, and the final version of this



(a) Loss vs. training FLOPs (log x).

(b) Loss vs. raw tokens seen (log x).

Figure 2: 125M comparison on native eval losses. On the FLOPs axis (left) the k=2 curve sits below the baseline curve along the whole trajectory at equal FLOPs; because the two curves score different position sets, the cross-k reading is validated separately on the full-position metric in Section 5.2. On the raw-token axis (right) the averaged model learns slightly *less* per token (averaging is lossy), but each of its tokens costs roughly half the compute. Stars mark Chinchilla-optimal reference points (D^*=20N).

paper will standardize on it for every cross-k claim. [TODO: re-run all final checkpoints through eval_full_positions.py and replace training-log endpoints where applicable]

5 Results

5.1 Iso-FLOP comparison on native losses: averaging ahead at every scale measured

A note on metric status before any numbers. The losses in this subsection are native eval losses: each model is scored at its own prediction positions, so the k=2 numbers average over roughly half the conditionals that the k=1 numbers do. Within a single curve, and between runs of the same k, these losses are exact; across k they are provisional readings whose validation requires the full-position metric of Section 5.2. That validation currently exists for the 125M pair and is queued for 50M and 250M. We present the native-loss picture first because it is the complete picture across scales, but the reader should weight the cross-k statements below accordingly.

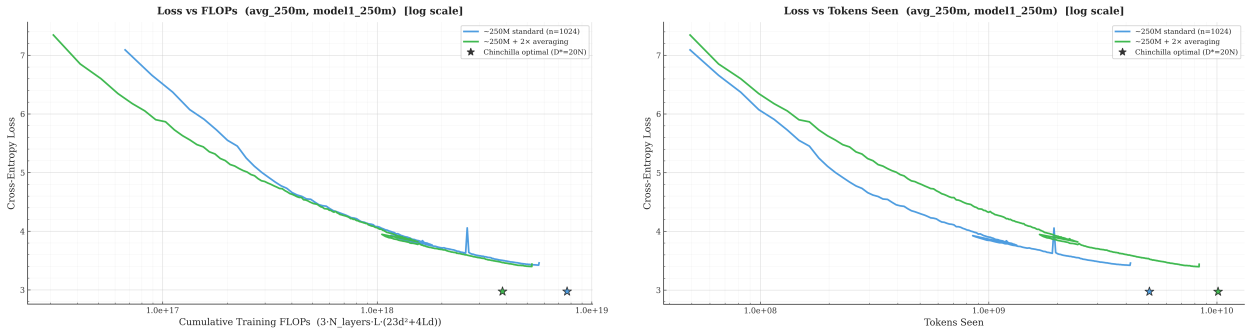
Figure 2 shows the 125M pair; Table 2 summarizes endpoints at all scales. Three observations.

First, the averaged model ends at or below the baseline's native loss at every scale measured, while spending fewer FLOPs at its endpoint (0.87× at 50M, 0.91× at 125M). The vertical gaps are modest (0.004 nats at 50M, 0.033 nats at 125M), but the horizontal reading is the operationally meaningful one: at 125M the baseline requires its full 1.50×10^{18} FLOPs to reach loss 4.204, whereas the averaged model crosses that loss at 1.05×10^{18}, a 1.44× compute saving on this metric; the same construction at 50M gives 1.15×. Late-training loss curves are extremely flat, so small vertical differences translate into large horizontal ones, which is also why we insist on reporting both readings rather than letting either flatter the method.

Second, the equal-loss multiplier is larger at 125M than at 50M (1.15× vs. 1.44×). We flag clearly what this is and is not: it is a trend in native-loss readings at two scale points, of which only the 125M point has so far been validated on the strictly comparable metric. It is not an established scaling law. The 250M pair (preliminary endpoints in Table 2; loss curves in Figure 3), full-position rescoring at all scales, and planned 500M/1B runs will determine whether the trend is real and

Table 2: Main iso-FLOP comparison (Config A, raw `seq_len` 1024). Losses are final native eval CE (nats/token) at each model's own prediction positions; cross-k readings are provisional pending full-position rescoring (done at 125M, Section 5.2). Train FLOPs are computed from tokens seen via Eq. (2) in all rows. The equal-loss saving is read horizontally off the native loss–FLOPs curves. 250M runs had not reached their target budgets at the time of writing.

Scale	Arm	Raw tokens	Train FLOPs	Final eval	vs. base	Saving
50M	k=1	1.0B	1.95×10^{17}	4.437	–	
50M	k=2	2.0B	1.70×10^{17}	4.433	0.87×	1.15×
125M	k=1	2.5B	1.50×10^{18}	4.204	–	
125M	k=2	5.0B	1.36×10^{18}	4.171	0.91×	1.44×
250M (prelim.)	k=1	4.18B / 5B	5.68×10^{18}	3.458	–	
250M (prelim.)	k=2	8.36B / 10B	5.26×10^{18}	3.438	–	*pending*



(a) Loss vs. training FLOPs (log x).

(b) Loss vs. raw tokens seen (log x).

Figure 3: 250M comparison on native eval losses (preliminary; both arms short of their target budgets: k=1 at 4.18B of 5B raw tokens, k=2 at 8.36B of 10B). The qualitative picture matches the smaller scales.

whether it continues, saturates, or reverses. The paper's eventual headline will be a Chinchilla-style fit of loss-versus-compute per family, on the full-position metric, with the efficiency multiplier as a function of scale. [TODO: full-position rescoring at 50M and 250M; fit $L(C)$ per family once 250M completes; add crossover analysis]

Third, the mechanism is visible in the raw-token view (Figure 2b): per raw token the averaged model learns *slightly less* than the baseline, since compression is genuinely lossy, but it processes each token at roughly half the compute, and the trade nets out positive. Token averaging does not make the model smarter per token; it makes tokens cheap enough that seeing twice as many wins.

5.2 Full-position evaluation at 125M: the gain survives a strictly comparable metric

The comparison above uses native eval losses, which Section 3.6 argued are not an apples-to-apples quantity across k. Table 3 therefore re-scores the final 125M checkpoints with the offset-ensemble protocol: identical held-out sequences, identical target positions, 3,409,119 predictions per model.

The averaged model is better by 0.035 nats, a 3.4% perplexity improvement, on a metric that scores every position exactly once from its full compressed prefix, while having spent 0.91× the

Table 3: Offset-ensemble full-position evaluation, 125M final checkpoints. Both rows average over the identical set of scored positions (every position $> k$ of each evaluation sequence); the baseline is restricted to exactly the position set covered by the k=2 ensemble.

Model	Mean NLL (nats/token)	PPL	Coverage
k=1 baseline (restricted)	4.177	65.1	all scored positions
k=2, 2-offset ensemble	**4.141**	**62.9**	all scored positions

training FLOPs. For this scale, the evaluation-comparability objection is answered on the merits, not argued around.

The per-offset decomposition contains the draft's most instructive surprise: offset 0 (the training grid) scores 4.145 and offset 1 scores 4.138. There is no off-distribution penalty at all, despite the model never having been explicitly trained at offset 1. As argued in Section 3.6, packed training data randomizes the grid relative to the text, so offset generalization was being trained all along. We draw the practical conclusions at their proper strength. For *this* checkpoint, the offset-ensemble metric is demonstrably valid despite fixed-offset training; the packing argument gives good reason to expect the same for our other checkpoints, and we will verify each as it is rescored rather than assume it. Likewise, the result makes rotating-offset generation plausible without retraining, but that scheme is a design at this point, not a measurement (Appendix B). If both expectations hold up, the pair (offset-ensemble evaluation, random-offset training) is what would make patch-style training viable as a *final* operating point rather than a warm-up.

5.3 Decomposition: cheaper tokens, not free context

Config A gains could in principle come from two entangled sources: more data per FLOP, or (in the Config B deployment) more raw context per prediction. The 50M decomposition experiment separates them. Four arms, all approximately iso-FLOPs except the deliberately more expensive full-attention control: the baseline (L=1024, 1024 raw context); Config B k=2 (`seq_len` 2048, L=1024, so 2× raw context at identical transformer cost); Config B k=4 (4096 raw context); and a k=1 model with true 2048-token attention (1.26× baseline cost).

Because native losses are only comparable within a fixed k and window scheme, we state the evidence as within-method comparisons, each of which is internally valid; the cross-k rows in Table 4 are shown for orientation only.

Table 4: Context-extension experiment at 50M, native eval losses. The Δ column gives the within-method comparison, which is the valid one: each extended-context arm against the same-k arm without extended context (k=1 full attention vs. the k=1 baseline; Config B k=2 vs. Config A k=2 at 4.433; Config B k=4 vs. Config A k=4 at 4.734). Losses in different k rows are not mutually comparable. The Config A comparators come from an earlier training batch (Appendix E).

Arm	Raw ctx	L	Raw tokens	Train FLOPs	Final eval (Δ within method)
k=1 baseline	1024	1024	1.00B	1.95×10^{17}	4.437
k=1 full attention	2048	2048	1.00B	2.45×10^{17}	4.651 (+0.214)
Config B, k=2	2048	1024	2.00B	1.95×10^{17}	4.650 (+0.217 vs. Config A)
Config B, k=4	4096	1024	4.07B	1.99×10^{17}	5.174 (+0.440 vs. Config A)

Table 4 and Figure 4 give the result, which is consistent across every valid comparison and

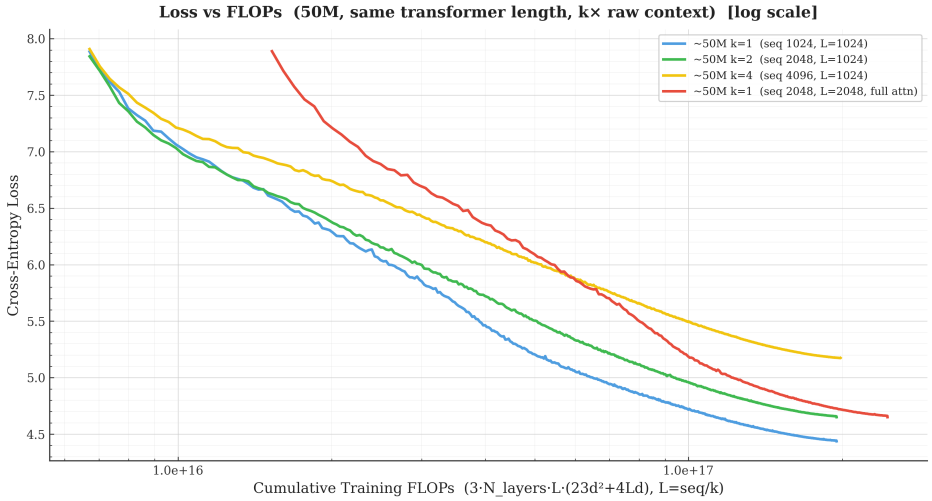


Figure 4: Context-extension experiment at 50M: native eval loss vs. training FLOPs (log x). Curves of different k score different position sets and are overlaid for orientation; the valid comparisons are within-method (see text). The two 2048-raw-context arms (green: k=2 averaging; red: full attention) end near the same native loss at very different cost; 4× context (yellow) is worse still. Early-training loss spikes are clipped at 8.0 for readability.

is, we think, one of the paper's more useful negative findings. **Extending raw context does not pay for itself at this scale, within either method.** Within k=1, widening the attention window from 1024 to 2048 costs +0.214 nats while also costing 1.26× the compute. Within k=2, spending the averaging saving on doubled raw context (Config B, 4.650) instead of pocketing it (Config A, 4.433) costs +0.217 nats on the same prediction task, with the caveat that the Config A run comes from an earlier training batch (Appendix E). Within k=4 the corresponding penalty grows to +0.440 nats. The failure is therefore a property of the data and scale, not of averaging: FineWeb documents are mostly shorter than 1024 tokens, and packed sequences cross document boundaries without masking, so context beyond 1024 consists largely of *other documents'* text, and the penalty compounds with the amount of extension. The corollary for Config A is exactly what we need: since added context is worthless to harmful here by every within-method reading, Config A's gains cannot be a context effect. On packed web text, the benefit of token averaging is data throughput per FLOP.

The experiment also contains a suggestive observation that we deliberately do not promote to a claim. The two 2048-context arms end at nearly identical *native* losses (4.6503 for k=2 averaging, 4.6505 for full attention), with the averaged route spending 1.26× less training compute, and with half the KV cache and attention cost at inference for the same raw context. If those native losses translate into equal full-sequence NLL, then averaging is a strictly cheaper way to buy raw context than widening the attention window. But the two numbers are measured on different position sets (one token per window versus all tokens), so their near-equality does not by itself establish equal language-modeling performance; this is precisely the comparability trap this paper is about, and we decline to fall into it in our own favor. Offset-ensemble rescoring of these two checkpoints is queued and will either confirm or retire the context-for-context reading. Either way, the question matters most on data where long context actually helps (long documents, code), which we flag as follow-up work. [TODO: offset-ensemble rescoring of both 2048-context arms; loss-vs-position curves; document-length distribution of FineWeb; PG19/code long-context experiment]

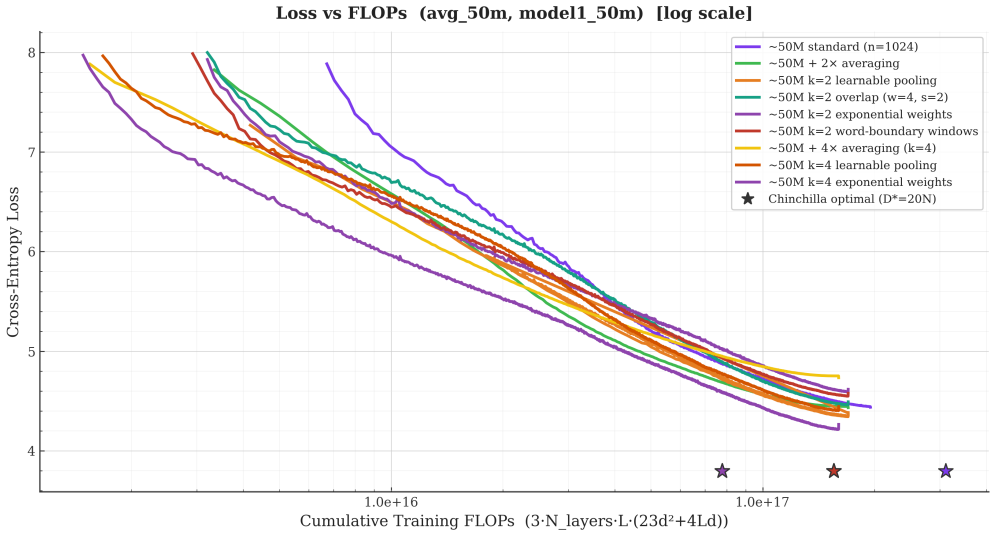


Figure 5: Pooling ablations at 50M: native eval loss vs. recomputed training FLOPs (log x), all k=2 and k=4 variants overlaid with the k=1 baseline and the original mean-pooling runs. Curves are mutually comparable only within a k block and matched protocol; see text.

5.4 Pooling ablations: among the rules tested, the best one depends on the compression ratio

Everything the transformer learns passes through the pooling function, so we ablate it at 50M under Config A, holding architecture, data, schedule, and token budgets fixed within each k (2.00B raw tokens at k=2; 4.07B at k=4). All runs within each k block share the same k and (except for the word-aligned variant) the same window scheme, so their native losses are mutually comparable. Figure 5 shows all trajectories; Table 5 the endpoints and quarter-budget checkpoints.

Two comparability boundaries apply and we draw them before interpreting anything. First, the "mean (original)" rows come from an earlier training protocol and are not matched controls (see the confounds paragraph below); no conclusion in this section compares against them, which also means *we make no claim yet about any variant versus plain mean pooling.* Matched mean reruns are training now. Second, the word-aligned variant predicts a different position distribution (word starts only) at a slightly different realized compression, so its native loss is not measured on the same prediction task as the fixed-window variants; we report it as indicative and exclude it from the strict ordering (Appendix C).

At k=2, the learned content-based rule leads. Among the three strictly comparable matched k=2 runs the ordering is stable throughout training: learnable < overlap < exponential. The content-based learned pooler (513 parameters, initialized at exact mean pooling) finishes 0.109 nats ahead of overlap and 0.234 ahead of the fixed recency rule. The recency prior actively hurts at k=2: with only two tokens per window, both carry useful signal, and systematically down-weighting the first discards it. Overlapping windows, which remove hard boundary cuts entirely, recover only part of the gap to learnable pooling, so boundary effects are not the dominant loss mechanism. The word-aligned run's numbers fall between overlap and exponential, but since it predicts a different position distribution we read it only as weak evidence that linguistically meaningful grouping is not automatically better than fixed-size grouping.

Table 5: Pooling ablations at 50M (Config A). Native eval CE (nats/token) at the nearest logged checkpoint to each quarter of the token budget. Runs are comparable within a k block under the matched protocol; the "mean (original)" rows use the earlier protocol and are for orientation only, and the word-aligned row is measured on a different prediction task (Appendix C).

k	Pooling	25%	50%	75%	Final	Final PPL
2	mean (original protocol)	5.061	4.638	4.480	4.433	84.2
2	**learnable (content)**	5.222	4.675	4.443	**4.383**	80.1
2	overlap (w=4, s=2)	5.460	4.820	4.562	4.492	89.3
2	word-aligned (task differs)	5.413	4.872	4.640	4.577	97.2
2	exponential (fixed)	5.439	4.962	4.707	4.617	101.2
4	mean (original protocol)	5.290	4.934	4.786	4.734	113.7
4	learnable (content)	5.315	4.753	4.512	4.445	85.2
4	**exponential (fixed)**	5.041	4.569	4.330	**4.265**	71.1

At k=4, the fixed recency rule beats the content-only learned scorer. The picture inverts at the harsher compression ratio, in the one matched pairing available there: the fixed exponential rule beats the learned content-based pooler by 0.274, 0.184, 0.182, and 0.180 nats at the four quarter-budget checkpoints. The gap is large, persistent, and present from early training, so it is not an endpoint artifact. We are careful about its scope: this is a comparison between these two specific poolers, not a general statement that positional priors dominate content at k=4. Our working explanation is architectural: the learned scorer evaluates each embedding *in isolation, before any positional signal exists in the network*, so it cannot express "the last token of the window matters most", which is exactly the prior that matters when four tokens collapse into one vector and the window's final token is the one adjacent to the prediction target. The fixed exponential rule hard-codes that prior. The natural synthesis, the content-plus-position pooler of Section 3.7 initialized at each k's best fixed rule, is running now and will test whether content information adds anything once the positional prior is expressible. [TODO: add learnable_pos results at k=2 and k=4]

Confounds we refuse to hide. The "mean (original)" rows in Table 5 came from an earlier training protocol and are *not* matched controls for the new runs: they used ∼8× fewer optimizer updates for the same token budget (∼131k vs. ∼16k raw tokens per update), a correspondingly different warmup exposure (∼262M vs. ∼33M warmup tokens), untied rather than tied embeddings, and a different evaluation batch count. Comparisons *within* the matched blocks are clean; the comparison of any new run against "mean (original)" is not, and we therefore do not yet claim that learnable or exponential pooling beats plain mean pooling. Matched mean reruns at both k under the exact new protocol are in flight, alongside the position-aware learned pooler and seed replicates; the final version of Table 5 will be internally matched throughout. What can be claimed today, cleanly, is the within-block result: among the rules tested, the best pooling is compression-dependent, with content-based weighting leading at k=2 and the fixed recency rule leading at k=4. [TODO: replace original-protocol mean rows with matched reruns; add seeds and variance bars; re-score all variants with offset-ensemble NLL]

6 Discussion and Limitations

Metric coverage. The paper's own standard is the full-position NLL, and at present that standard is met only at 125M. The 50M and 250M cross-k comparisons, the scale trend of the efficiency multiplier, and the context-for-context observation all currently rest on native losses and are queued for rescoring. We have organized the draft so that every such claim is labeled with its metric; the final version should have no cross-k claim on native losses at all.

Scale. Two scale points (plus a preliminary third) support the efficiency trend on native losses; they do not establish a scaling law. The 250M pair must finish, and 500M/1B runs are required before any claim about the trend's continuation. Data logistics are part of this: at k=2 the averaged arm consumes twice the raw tokens, and the 250M k=2 run already exhausts FineWeb sample-10BT, so larger runs move to a $\geq$100BT slice. That introduces a data-distribution change between scales that we will need to handle, either by rerunning smaller scales on the larger slice or by fitting per slice.

Optimizer-step confound. Iso-FLOP comparisons at $k\times$ tokens inherently give the averaged arm more optimizer updates at fixed batch size. A k=1 control with halved batch size and doubled steps (same FLOPs, same update count as the k=2 arm) is queued; without it, part of the headline gain could in principle be an update-count effect rather than a data effect. The pooling-ablation experience (Section 5.4) demonstrates concretely that update count moves endpoints at these scales, which is why we treat this control as mandatory rather than optional.

Tokenizer-granularity control. Averaging k=2 tokens resembles, at a squint, simply using a tokenizer with 2$\times$ longer average tokens. The two are not the same, since averaging preserves the fine vocabulary and composes embeddings dynamically, but a coarser-tokenizer baseline compared in bits-per-byte is the honest control and is planned.

Generation and downstream evaluation. Everything reported here is language-modeling loss. Rotating-offset decoding is a design (Appendix B) that the offset-robustness result makes plausible, but it has not been implemented or benchmarked, and downstream zero-shot quality (LAMBADA, HellaSwag, PIQA, ARC-E) remains to be measured. A small local decoder that emits all k tokens per window (Block-Transformer-style) is a natural extension that would also give native generation. [TODO: implement rotating-offset decoding; downstream evals]

Data dependence of the context result. The finding that extended raw context hurts is a statement about packed FineWeb at 50M, where most documents are shorter than 1024 tokens and packing crosses boundaries without masking. It is not a claim about long-document corpora, code, or masked packing, where the suggestive context-for-context advantage of averaging over full attention, if it survives rescoring, could translate into a genuine long-context win. We consider this the most promising unexplored branch of the work.

Residual inconsistencies. The 50M main pair is untied while larger scales are tied; the pooling ablations' matched mean controls are still training; the Config A comparators in the context experiment come from an earlier training batch than the Config B arms; the word-aligned variant's metric differs by construction; offset robustness is verified at one checkpoint and assumed (with

stated reasons) elsewhere; and all pre-existing checkpoints were trained before random-offset training landed. We list these here so that no reviewer has to discover them.

7 Conclusion

[TODO: Write after 250M completes, full-position rescoring lands at all scales, and the matched pooling controls finish. Skeleton: (i) token averaging turns surplus data into lower loss at fixed compute, verified on the strictly comparable full-sequence metric at 125M (3.4% perplexity at 0.91× FLOPs), with native-loss multipliers of 1.15× (50M) and 1.44× (125M) pending rescoring; (ii) offset-ensemble evaluation and random-offset training resolve the comparability problem, and, if rotating-offset decoding benchmarks well, make patch-style operation viable as a final operating point; (iii) within each method, extended raw context hurts on packed web text, so the gain is data-per-FLOP; the context-for-context advantage of averaging is suggestive pending rescoring; (iv) among the pooling rules tested, the best choice depends on the compression ratio, and a position-aware learned pooler plausibly subsumes the regimes. Close with the scaling question as the decisive open item.]

A FLOPs Derivation

Per layer and per transformer position, counting multiply–accumulates as 2 FLOPs, forward pass:

- Attention projections: Q, K, V, and output, i.e. four $d \times d$ matmuls: $4 \times 2d^2 = 8d^2$.
- SwiGLU FFN with `ff_multiplier` 2.5: the combined implementation projects $d \to 5d$ for up and gate ($2 \cdot 5d^2 = 10d^2$), then projects down $2.5d \to d$ ($2 \times 2.5d^2 = 5d^2$): total $15d^2$. (A standard 4× FFN would give $16d^2$; our architecture's SwiGLU at 2.5 gives $15d^2$, and we use the exact value.)
- Attention scores and value aggregation: $2 \cdot 2Ld = 4Ld$ per position at sequence length L (causal masking halves the effective term; we follow the standard convention of quoting the full-matrix count and apply it uniformly to all arms, so ratios are unaffected). [TODO: state the causal-mask convention once and verify consistency with plotting code]

Total forward: $23d^2 + 4Ld$ per layer-position. Training multiplies by 3 (forward + backward). A sequence of `seq_len` raw tokens occupies $L = \texttt{seq\_len}/k$ positions, giving Eq. (2). We recompute FLOPs from `tokens_seen` via Eq. (2) in every plot and table of this paper, including the preliminary 250M endpoints; the `cumulative_flops` column logged by early training code was inconsistent across runs and is never used.

Embedding and output-head FLOPs are excluded from Eq. (2) on both arms; at these scales the head contributes a few percent and equally to all arms of a comparison. [TODO: quantify head share at 50M for completeness]

B Offset-Ensemble Evaluation: Details

For a k-averaged model and evaluation sequence $t_{1..T}$, offset $o \in \{0, \ldots, k-1\}$ discards the first o tokens, forms $n_o = \lfloor (T-o)/k \rfloor$ windows, embeds and pools them, and runs the transformer on the first $n_o - 1$ compressed positions. The prediction at compressed position j (1-indexed) is scored against the raw token at position $o + jk + 1$. Across offsets, the predicted position sets are disjoint

and their union is every position $> k$ (up to edge effects at the sequence tail), each predicted exactly once from its full compressed prefix. Mean NLL is computed by pooling all scored predictions across offsets and sequences. For the restricted baseline, the k=1 model's per-position NLLs are gathered on exactly this position set. Total cost is k forward passes at length $\approx T/k$, i.e. approximately one baseline pass at length T (the attention term is actually cheaper by a factor k).

Generation would rotate the anchor: to emit the token at raw position p+1 given a compressed cache for some offset, the decoder uses the offset $o \equiv p \bmod k$ whose window grid ends exactly at p. Maintaining k caches (one per residue class) amortizes to approximately one baseline decoder's cost, since each cache is $1/k$ the length. This scheme is a design; it has not yet been implemented or benchmarked, and until it is, generation claims in this paper should be read as architectural arguments. [TODO: implement and benchmark rotating-offset decoding; measure downstream tasks]

C Word-Aligned Windows: Algorithm and Caveats

A token begins a word iff its string form starts with the BPE whitespace marker (a leading `Ġ` in GPT-NeoX byte-level BPE) or a newline marker, or is a special token; a boolean lookup table over the padded vocabulary is built once from the tokenizer. Grouping is greedy per sequence: accumulate tokens into the current window; close the window when it holds $\geq k$ tokens *and* the next token starts a new word, or unconditionally at $2k$ tokens (so a pathologically long word cannot grow a window without bound); drop a trailing partial window. Windows therefore span 2–4 tokens at k=2, never end mid-word (unless a word exceeds $2k$ tokens), and realized compression on FineWeb is $\sim$2.1$\times$ at k=2. Batch handling truncates all sequences in a batch to the minimum window count; the grouping loop costs $\sim$1ms per batch at B=16, T=1024 (negligible). Targets are the first token of the next window, which for this variant is always a word-start token.

Two comparability caveats follow. First, realized compression exceeds the nominal 2$\times$, so iso-FLOP accounting must use the realized ratio (the transformer sees $\sim$5% fewer positions per raw token than uniform k=2). Second, the predicted-position distribution differs (word starts only), so its native eval loss is not measured on the same prediction task as the uniform variants; the offset-ensemble protocol does not directly apply to variable-size windows either. A bits-per-byte comparison, or a fixed-position rescoring, is required before strong claims; we present the word-aligned row as indicative only. [TODO: bits-per-byte rescoring for the word variant]

D Pooling Modules

Content-based learnable pooler. Shared scorer $s_i = w^\top e_i + b$ with $w \in \mathbb{R}^{d_{\text{model}}}$, $b \in \mathbb{R}$, zero-initialized so that $\alpha_i = 1/k$ exactly at initialization (training starts at mean pooling); $\alpha = \text{softmax}(s_{1..k})$ within each window; $\bar{e} = \sum \alpha_i e_i$. Parameters: $d_{\text{model}} + 1 = 513$. Trained jointly with the LM by the ordinary objective; no auxiliary reconstruction loss is used in these experiments.

Position-aware learnable pooler. $s_i = w^\top e_i + b + p_i$ with $p \in \mathbb{R}^k$ a learned per-slot bias. Initialization: $p = 0$ (uniform) at k=2, where the recency prior hurt; $p_i \propto$ the exponential profile with last-to-first ratio 20 at k=4, where it won. Parameters: $d_{\text{model}} + 1 + k$. Both terms train end-to-end; inspecting the learned α_i by position and token type is part of the planned analysis. [TODO: report learned weight profiles]

Fixed weighting schemes. $w_i \propto \exp(\alpha i)$ with $\alpha = \ln(20)/(k-1)$ (exponential; the trained representative), plus linear, Gaussian, and triangular alternatives implemented but not trained at

scale. All schemes are L_1-normalized.

E Full Run Table

Table 6: All runs referenced in this draft, with canonical result files. "Protocol" distinguishes the original training protocol (large per-update token count) from the matched ablation protocol (batch 16 × `seq_len` 1024 per update, 16 eval batches, tied embeddings). The Config B context arms and the k=1 context controls were trained in a later batch than the original-protocol Config A runs; the within-k=2 and within-k=4 context comparisons in Section 5.3 therefore cross training batches, as flagged there. [TODO: add exact batch size, world size, and step counts per run; add seeds; update as matched controls and 250M complete.]

Run	k	Raw tokens	Final eval	Protocol	Notes
50M k=1 (clean rerun)	1	1.0B	4.437	original	untied; canonical baseline
50M k=2 mean	2	2.0B	4.433	original	untied; canonical
50M k=4 mean	4	4.07B	4.734	original	untied; canonical
50M k=1, 2048 full attn	1	1.0B	4.651	original	context control
50M k=2, seq 2048 (Config B)	2	2.0B	4.650	original	context arm
50M k=4, seq 4096 (Config B)	4	4.07B	5.174	original	context arm
125M k=1	1	2.5B	4.204	original	tied
125M k=2	2	5.0B	4.171	original	tied
250M k=1	1	4.18B/5B	3.458	original	tied; incomplete
250M k=2	2	8.36B/10B	3.438	original	tied; incomplete
50M k=2 learnable	2	2.0B	4.383	matched	
50M k=2 overlap $w4s2$	2	2.0B	4.492	matched	
50M k=2 word-aligned	2	2.0B	4.577	matched	metric caveats
50M k=2 exponential	2	2.0B	4.617	matched	
50M k=4 learnable	4	4.07B	4.445	matched	
50M k=4 exponential	4	4.07B	4.265	matched	
50M k=2/k=4 mean (matched)	2/4	2.0B/4.07B	*running*	matched	control reruns
50M k=2/k=4 learnable+pos	2/4	2.0B/4.07B	*running*	matched	

F Training Instabilities and Excluded Runs

For completeness and reproducibility we document the detours.

1. **RoPE bug.** An early implementation bug in the rotary embedding invalidated the first tied-embedding run and several early k-sweep results. All affected logs are quarantined (`*_rope_bug` suffixes in the results tree) and no number in this draft derives from them.

2. **50M baseline instability.** The original 50M k=1 run suffered an early loss explosion and converged ∼1 nat too high; a clean rerun (the 4.437 baseline used throughout) matches the cross-scale pattern. All runs at this scale show a large transient loss spike in the first ∼2k steps that self-recovers; it is an initialization artifact, clipped in plots.

3. **The seq_len accident.** As noted in Section 3.4, Config A resulted from launching k=2 with the baseline's raw `seq_len`; it became the paper's primary configuration and the intended setting became Config B.

4. **Duplicated ablation log.** The k=2 learnable run's CSV contains two interleaved logging streams and one malformed line (from a resumed job double-writing); the two recorded

endpoints agree to 0.004 nats (4.379 vs. 4.383) and the file is cleaned deterministically before plotting.

G Hyperparameters

Table 7: Shared hyperparameters. [TODO: fill exact per-run batch sizes and world sizes from run logs.]

Optimizer	AdamW
LR schedule	cosine, decay to 10% of peak over total steps
Warmup	2,000 steps
Peak LR	$2.0/1.6/1.4 \times 10^{-4}$ (50M/125M/250M)
Precision	bfloat16 autocast
Raw `seq_len`	1024 (Config A; Config B as stated)
Tokenizer	EleutherAI Pythia-70m (GPT-NeoX BPE), vocab 50,257
Vocabulary padding	50,304
FFN	SwiGLU, expansion 2.5
Positional encoding	RoPE (applied to compressed positions)
Chinchilla reference constants	A=406.4, α=0.3392, B=410.7, β=0.2849, E=1.6934